

Activity Report

Course: Computational Intelligence

Academic Year: 2024-2025

Student: Stefano Roy Bisignano

ID: s333971

1. Introduction

This report documents the activities completed during the *Computational Intelligence* course in the academic year 2024-2025. The course covered topics such as local search, evolutionary algorithms, and multi-agent systems, applied in laboratory exercises and the final project. The report includes details on laboratory results, feedback received, and data, along with a comprehensive description of the symbolic regression project.

2. Laboratory Activities

Lab 0: Irony.md

- **Objective:** Create a GitHub repository and upload a file named `irony.md` containing a humorous comment about the first lecture.
 - **Collaboration:** Individual work.
-

Lab 1: Hill Climbing for the Set Cover Problem

- **Objective:** Solve the *Set Cover Problem* using a technique discussed in class.
- **Collaboration:** Individual work.
- **Techniques Used:** Hill Climbing algorithm based on single mutations in each iteration.

Key Results

- **Full coverage** achieved in all tested instances.

Criticisms Received

- The code lacked explicit solution validation; the `valid()` function was never called. This was highlighted in a peer review by @carlopantax,

who suggested incorporating a combined fitness function to verify both coverage and cost.

Feedback Given

- Emphasized the importance of documenting key functions like `tweak_RMHC` and adding graphs to visualize cost evolution over time in other peer reviews.

Lesson Learned Documentation and solution validation are critical for ensuring code clarity and reliability.

Lab 2: Travelling Salesman Problem (TSP)

- **Objective:** Develop and compare two algorithms for solving the TSP.
- **Collaboration:** Individual work.
- **Techniques Used:**
 1. **Genetic Algorithm (GA):** Based on selection, crossover, and mutation.
 2. **Greedy Algorithm:** A fast heuristic solution.

Key Results

- **Small datasets:** GA outperformed the greedy approach in terms of total cost.
- **Large datasets:** The greedy algorithm proved more efficient, achieving significantly lower costs.

Criticisms Received

- @mericuluca suggested optimizing data management by introducing parallelization and caching of distance matrices to improve computation speed.
- @FruttoCheap recommended adopting an **adaptive mutation approach** to reduce stagnation during evolution.

Feedback Given

- Proposed visualizing the cost convergence of GA compared to the greedy algorithm to highlight performance differences.

Lesson Learned Parallelization and data structure optimization are essential to enhance algorithm efficiency on large datasets.

Lab 3: Solver for Puzzles Using Bidirectional A*

- **Objective:** Explore a bidirectional A* algorithm to solve NxN puzzles, using Manhattan distance as the heuristic.
- **Collaboration:** Individual work.
- **Techniques Used:** Bidirectional A* algorithm with Manhattan distance.

Key Results

- **3x3 Puzzle:** Solution found in 22 steps, with an efficiency (quality/cost) of 0.074.
- **4x4 Puzzle:** Solution found in 52 steps, but with a high computational cost (276,746 nodes explored).

Criticisms Received

- @imEmaa suggested comparing the bidirectional approach with classic A* to demonstrate actual efficiency improvements.
- @Gabry323387 recommended integrating the linear conflict heuristic to further reduce the number of nodes explored.

Feedback Given

- Commended a peer's implementation for consistently generating valid solutions, while suggesting improved graph documentation for better accessibility.

Lesson Learned Incorporating advanced heuristics can significantly enhance performance in search problems.

3. Final Project: Symbolic Regression

Collaboration

In collaboration with Mirko Di Maggio, we developed a modular framework for symbolic regression, evolving it from pure genetic programming to a memetic approach. This approach combines the strengths of genetic programming with local search techniques, significantly improving the system's performance and efficiency.

3.1 Introduction

The symbolic regression project aims to develop a system capable of generating interpretable mathematical formulas that describe the relationship between input variables (X) and output (Y). This challenge in computational intelligence focuses on optimizing such formulas to minimize the mean squared error (MSE)

on training and testing data while maintaining computational efficiency and NumPy compatibility.

3.2 Methodology

3.2.1 General Approach The core of the project is genetic programming (GP), which evolves syntax trees representing mathematical formulas. The evolutionary process includes the following steps for each generation:

1. **Evaluation:** Fitness of each individual is calculated based on **MSE** and tree complexity.
2. **Selection:** Parents are chosen using strategies like elitism or tournaments.
3. **Crossover:** Subtrees are exchanged between parents to generate new individuals.
4. **Mutation:** Random changes are introduced to explore new solutions.
5. **Local Search:** Solutions are refined using techniques such as hill climbing or simulated annealing.

3.2.2 Parameters and Configuration The system configuration was optimized to balance exploration and exploitation using the following parameters:

- **Population size:** 600, increased to ensure greater genetic diversity.
- **Maximum tree depth:** 7, reduced to mitigate bloat.
- **Number of generations:** 50, increased to allow more evolution.
- **Tournament size:** 10, set to apply higher selection pressure.
- **Mutation rate:** 0.6, reduced to balance the effect of crossover.
- **Crossover rate:** 0.4, increased to encourage exploration.
- **Elitism:** 10, with more elite individuals preserved.
- **Tree complexity penalty:** 0, applied to favor simpler trees and reduce overfitting.
- **Local search:** Enabled to improve solutions during evolution.
- **Adaptive strategies:** Enabled to dynamically balance exploration and exploitation.
- **Early stopping:** 20 generations without improvements.
- **Fitness threshold:** 1, as the minimum objective for fitness.
- **Seed:** 42, to ensure reproducibility of experiments.

3.2.3 Implementation

Formula Representation Formulas are represented as syntax trees where:

- **Internal nodes:** Represent mathematical operators (e.g., $+$, $\sin$, $\log$).
- **Leaf nodes:** Represent constants or variables.

Operator Selection and Implementation Mathematical operators were assigned relative costs based on:

- **Average execution times:** Measured using benchmarks on large arrays (10^6 elements).
 - **Operator arity:** Distinction between unary and binary operators. Binary operators, while not directly affecting computational time, increase tree complexity and are penalized to favor simpler trees and reduce overfitting.
-

3.3 System Workflow

The system workflow is divided into the following steps:

1. **Data Loading:** `.npz` files contain input (X) and output (Y) datasets.
 2. **Initial Population Generation:** Individuals are randomly generated with limited tree depth.
 3. **Evolution:** For each generation:
 - Fitness is calculated.
 - Selection, crossover, and mutation are applied.
 - Solutions are refined through local search.
 4. **Result Saving:** The best formula is saved in a Python file (`s333971.py`).
 5. **Visualization:** Graphs are generated for fitness and diversity analysis in each experiment.
-

4. System Modules

The project is organized into multiple modules to separate functionalities and facilitate maintenance.

4.1 Core Module

Handles the main components of genetic programming: - **tree.py:** Manages syntax tree representation. - **safe_math.py:** Implements safe mathematical operations to handle numerical errors. - **evaluator.py:** Calculates MSE and tree fitness. - **statistics.py:** Collects statistics during evolution.

4.2 Memetic Module

Implements local search techniques and evolutionary management: - **local_search.py:** Includes strategies like hill climbing, tabu search, and simulated annealing. - **mutation.py:** Manages adaptive mutation strategies. - **crossover.py:** Applies crossover strategies. - **selection.py:** Selects parents using strategies like tournaments or elitism.

4.3 Utility Module

Supports auxiliary functionalities: - **plotting.py**: Generates graphs for fitness, diversity, and complexity analysis. - **logger.py**: Records detailed logs for each experiment. - **utils.py**: Contains generic functions like data loading and experiment initialization.

4.4 Configuration and Main Script

- **memetic_config.py**: Contains all configurable system parameters.
 - **main.py**: Entry point for experiments, managing data loading, population evolution, and result saving.
-

5. Future Prospects

- **Reinforcement Learning Integration**: To guide strategy evolution more intelligently and dynamically.
 - **Advanced Regularization Techniques**: To improve the generalization capability of formulas.
 - **Advanced Heuristics Expansion**: To adopt more complex heuristics for local search.
 - **Scalability Optimization**: To enhance large dataset management through parallelization and advanced caching.
-

6. Declaration of Originality

I declare that this work is original and that all sources used have been duly cited. The developed code is my own, created in collaboration with Mirko Di Maggio, and does not violate third-party rights. Any external contributions have been acknowledged in the report.

7. Appendix: GitHub Repositories

- Lab 0
 - Lab 1
 - Lab 2
 - Lab 3
 - Symbolic Regression Project
-